DexCompose: Reusing Dexterous Policies for Multi-Task Manipulation with a Single Hand

Anonymous Author(s)
Affiliation
Address
email

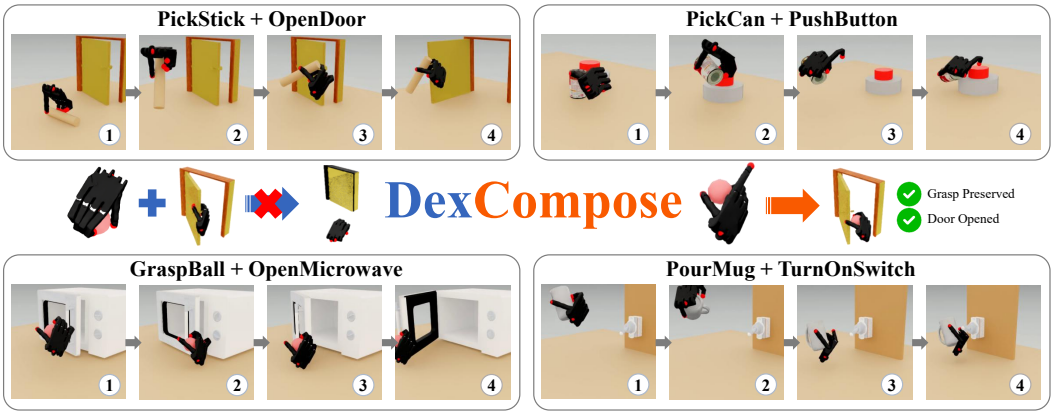

Figure 1: DexCompose composes dexterous skills through role-aware finger ownership. By separating grasp preservation from downstream interaction with asymmetric residuals, the framework reduces destructive interference and enables robust multi-skill manipulation.

Abstract: Dexterous manipulation policies can solve individual skills, but composing them to perform multiple tasks with a single hand remains difficult. Different tasks often compete over the same action dimensions, causing destructive interference between preserving an existing manipulation outcome and executing a new one. We propose DexCompose, a role-aware residual composition framework that enables the reuse of pretrained dexterous policies through explicit finger-level action ownership. Given two pretrained full-hand policies, DexCompose first collects successful post-task states from the first skill and performs release tests over candidate finger masks to estimate which fingers are necessary for maintaining the established held state. It then trains a bounded residual stabilizer for task preservation and a context-aware residual that adapts the frozen downstream policy only within the action subspace assigned to the new task. We evaluate the framework on 16 composite dexterous manipulation tasks spanning four object-retention skills and four downstream interactions. Our method achieves 77.4% average composite success, and ablation studies confirm that structural action ownership combined with dual asymmetric residuals is more effective than conventional policy chaining or unmasked residual correction.

Keywords: Dexterous Manipulation, Policy Composition

1 Introduction

Dexterous manipulation often requires a hand to preserve the outcome of one skill while executing another. For example, after grasping a stick, a robot may need to use the same hand to push a door handle without dropping the object. Although grasping and pushing behaviors can each be learned by separate single-task policies, directly chaining these policies is unreliable: the downstream policy controls the full hand and may overwrite the finger motions required to maintain the grasp. This

Submitted to the 10th Conference on Robot Learning (CoRL 2026). Do not distribute.

creates interference between two objectives that share the same action space: preserving the grasp while performing the new interaction task. A straightforward alternative is to train a separate policy for every composite task, but this scales poorly, requiring $n \times m$ policies for n grasp-maintenance skills and m downstream interactions. These limitations motivate a compositional approach that reuses pretrained single-task policies while explicitly managing how finger actions are allocated across objectives.

Recent work has shown the possibility of leveraging selective hand degrees-of-freedom (DoF) allocation to enable multiple dexterous interactions with single hand. MultiGrasp [1] studies multi-object grasping by generating multi-object grasp proposals and learning a policy to execute them. Beyond grasping, DexMulti [2] enables concurrent grasping and manipulation by decomposing interactions into reusable object-centric skills. HANDFUL [3] learns resource-aware grasps by explicitly reserving fingers for downstream manipulation subtasks. These methods suggest that complex dexterous interactions can be achieved by allocating hand resources across different manipulation objectives. However, existing finger-aware approaches typically rely on predefined finger allocations or downstream task compositions during grasp generation, data construction, or policy training, making them difficult to scale across different task combinations. Meanwhile, large-scale dexterous grasp datasets such as DexGraspNet, UniDexGrasp, and UniDexGrasp++ [4, 5, 6] have made reusable full-hand grasp controllers increasingly available. This motivates a question: *can we compose pretrained dexterous manipulation policies by discovering and exploiting redundant hand DoFs across tasks?*

Under this perspective, we reframe task combinations as a resource-allocation problem at the embodiment level. The key insight is that idle DoFs within a trained policy are a reusable resource for composing additional behaviors. Our approach should identify which action dimensions are necessary for preserving the first task's outcome, assign structural ownership of those dimensions, and restrict each subsequent policy to operate only within its allocated subspace. This reframing decomposes the problem into two sub-problems: (1) **discovery**, identifying which action dimensions each policy actually needs, and (2) **composition**, chaining different policies by ensuring each policy acts only through its allocated DoFs so that their objectives can be realized concurrently.

We propose DexCompose, a two-stage pipeline that addresses both sub-problems. For discovery, we introduce **finger attribution**: we collect successful held-object states from the first task and identify finger-level redundancy through release tests. Specifically, we evaluate different finger masks by releasing subsets of fingers and observing whether the object remains stably retained. This process reveals which fingers are essential for maintaining the grasp. The task-specific mask is then selected to balance grasp stability with the dexterity needed for the downstream task. For composition, we introduce **dual residual stabilizer**: one lightweight residual module preserves the held-object configuration against disturbances caused by base motion and redundant finger release, while the other provides task-aware compensation for the downstream policy under the motion constraints imposed by the maintained grasp.

We evaluate DexCompose on 16 composite manipulation tasks spanning four hold-and-retain tasks and four downstream interactions. Our method achieves 77.4% average composite success, outperforming direct policy chaining by a 74.3 percentage points and the strongest baseline by 15.8 percentage points. Results show that the dual residual stabilizer is the primary factor enabling successful composition, as it maintains stable grasp retention while adapting downstream actions under the constraints imposed by the maintained grasp. Finger allocation further improves composite success under the same training budget by reducing cross-task interference through structured action ownership.

In summary, our main contributions are as follows.

- We introduce a post-hoc composition framework for pretrained dexterous manipulation policies, formulating composite manipulation as an action allocation problem where grasp preservation and downstream interaction must share the same multi-fingered hand.

- We propose DexCompose, a role-aware residual pipeline that combines training-free finger attribution, explicit action ownership, and dual residual stabilization to compose two pretrained full-hand policies without training full policies for each task pair.
- We evaluate DexCompose on 16 composite manipulation tasks and show that it substantially improves composite success over policy chaining and unmasked residual baselines, highlighting a scalable path toward reusable dexterous manipulation systems that can flexibly combine learned skills.

2 Related Work

Dexterous policy composition. Prior work on dexterous policy composition mainly studies three directions: learning reusable dexterous controllers, sequencing them for long-horizon manipulation, and allocating fingers to enable multi-object interactions with a single dexterous hand. A large body of work learns robust reusable dexterous policy via reinforcement learning, imitation learning, and generative or diffusion policies [7, 8, 9, 10, 11, 12, 13, 14, 15, 16]. Multiple benchmarks and grasp-generation works scale dexterous manipulation across objects, articulated categories, cluttered scenes, and hand morphologies [17, 18, 5, 19]. The second line explicitly studies dexterous skill sequencing. SequentialDexterity [20] chains multiple dexterous policies by learning transition feasibility and selecting compatible policies across interaction stages. Related long-horizon manipulation approaches also leverage skill priors, behavior primitives, or retrieval from prior data to reduce exploration and reuse previous experience [21, 22, 23, 24]. The third direction exploits multi-finger hands for multi-object or multifunctional manipulation. MultiGrasp [1], SeqMultiGrasp [25], and SeqDiffuser [26] study multi-object grasp acquisition, while DexMulti [2] and HANDFUL [3] focus on multi-stage dexterous manipulation and future-aware grasping. In contrast, our work approaches dexterous manipulation from the perspective of pretrained policy reuse, enabling the composition of two policies at inference time without modifying the original controllers.

Residual policy learning. Residual policy learning improves an existing controller by learning an additive correction on top of a prior action. Early residual formulations combine an imperfect policy with a learned residual, improving robustness under long horizons, partial observability, model mismatch, and sparse rewards. [27, 28, 29]. Residual reinforcement learning further applies this idea to robot control by superimposing a learned residual action on a trained controller, where the controller governs the primary behavior while RL compensates for modeling errors and complex contact dynamics [30]. Subsequent work improves upon priors by incorporating demonstrations, human commands, learned skills, or imitation policies, enabling residual learning for a wide range of robotic manipulation tasks, including shared autonomy, deformable-object manipulation, and precise assembly refinement. [31, 32, 33, 34, 35]. More broadly, skill-prior and behavior-prior methods learn reusable latent spaces or primitive libraries that guide downstream policy learning [21, 22, 23, 24], while recent dexterous transfer work also uses residual modules to adapt pretrained motion priors to high-DoF bimanual manipulation [36]. Different from these works, we formulate the residual component as a stabilizer that compensates for potential failure modes of the policy under unseen scenarios, improving robustness against distribution shifts and unexpected interactions.

3 Method

In this section, we introduce the framework of DexCompose. We first formulate the problem of reusing pretrained policies for composed manipulation tasks (Sec. 3.1). We then decompose the problem into two subproblems. The first is to discover and allocate finger-level ownership masks that determine which fingers can be reassigned to the downstream task while preserving the outcome established by the pretrained policy (Sec. 3.2). After obtaining the allocation mask, we compose the two tasks using a dual-residual stabilization framework, where one residual module maintains stability for the preserved fingers and the other adapts the downstream policy within its assigned action subspace (Sec. 3.3). Fig. 2 summarizes the overall pipeline.

3.1 Problem Formulation

We consider sequential dexterous manipulation with two pretrained diffusion policies, π_A and π_B. Policy π_A executes an initial skill whose physical outcome must be preserved, such as retaining a

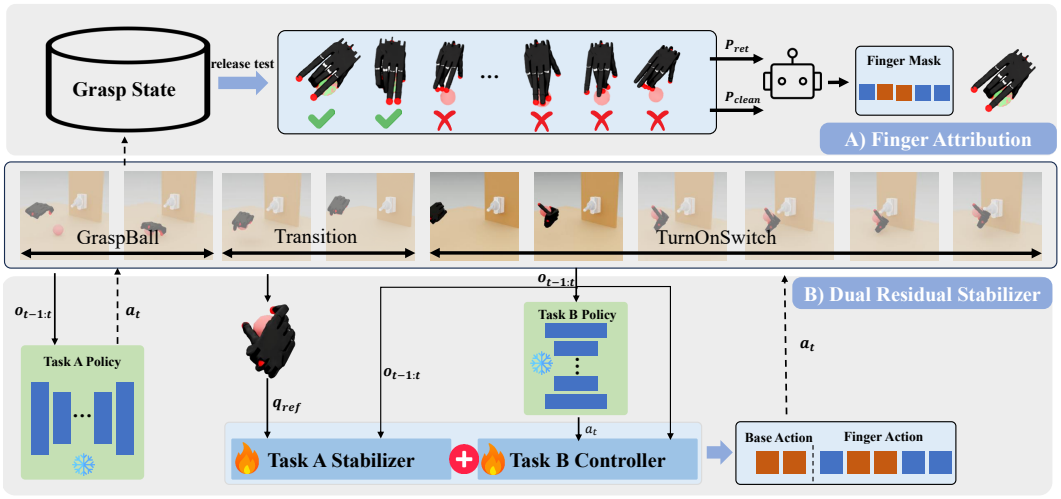

Figure 2: Overview of DexCompose. Given two frozen single-task policies, we first attribute fingers according to their necessity for preserving the first task and their availability for the downstream task. The selected finger-level mask is then expanded into action-space masks. During execution, a dual-residual stabilizer preserves the first task on its assigned fingers while adapting the downstream policy on the wrist and released fingers.

grasped object, while policy π_B executes a downstream interaction skill. The goal is to perform Task B while maintaining the outcome established by Task A.

At time step t, let o_t denote the policy observation and let the action be $a_t = (p_t, r_t, q_t) \in \mathbb{R}^d$, where $p_t \in \mathbb{R}^3$ is the wrist position, $r_t \in \mathbb{R}^3$ denotes the wrist rotation represented in euler angles, and $q_t \in \mathbb{R}^{d_q}$ denotes the hand joint values. Thus, the action dimension is $d = 3 + 3 + d_q$.

The two frozen policies output actions in the same action space, i.e., $a_t^A = \pi_A(o_t)$ and $a_t^B = \pi_B(o_t)$. We keep both pretrained policies fixed and learn lightweight composition models parameterized by Θ to produce the composed action

$$a_t^{\text{comp}} = F_\Theta(o_t, a_t^A, a_t^B). \tag{1}$$

A composed rollout is initialized from a successful Task-A state s_0 and follows

$$\tau = (s_0, a_0^{\text{comp}}, s_1, a_1^{\text{comp}}, \ldots, s_H). \tag{2}$$

Let $S_A(\tau)$ indicate whether the outcome of Task A is preserved throughout the rollout, and $S_B(\tau)$ indicate whether Task B succeeds. The objective of policy composition is then

$$\max_\Theta \ \mathbb{E}_{\tau \sim \pi_{\text{comp},\Theta}} \left[S_A(\tau) S_B(\tau) \right]. \tag{3}$$

3.2 Finger Attribution

To reduce interference between the two policies, we assign a subset of fingers to preserve the Task-A grasp while releasing the remaining fingers for Task B. Let $\mathcal{F} = \{1, \ldots, F\}$ denote the finger set ($F = 5$), and let $m \in \{0,1\}^F$ be a binary ownership mask, where $m_f = 1$ indicates that finger f is preserved for Task A and $m_f = 0$ indicates that it is released for Task B. The finger-level mask is expanded to a joint-level mask $\bar{m} \in \{0,1\}^{d_q}$ over the hand joints. During policy composition, Task A only controls the preserved finger joints, while Task B controls the released finger joints together with the wrist DoFs.

To identify releasable fingers, we collect successful post-grasp states after Task A execution.

$$\mathcal{D}_{\text{hold}} = \{(s_i^{\text{hold}}, q_i^{\text{ref}})\}_{i=1}^N. \tag{4}$$

For each candidate mask m, preserved fingers replay the reference grasp while released fingers are gradually opened:

$$q_{i,t}^{\text{test}} = \bar{m} \odot q_i^{\text{ref}} + (1 - \bar{m}) \odot \left[(1 - \alpha_t) q_i^{\text{ref}} + \alpha_t q^{\text{open}} \right]. \tag{5}$$

We evaluate each mask using two metrics: the retention rate $P_{\text{ret}}(m)$, which measures whether Task A remains successful throughout the release test, and the clean-release rate $P_{\text{clean}}(m)$, which measures whether released fingers disengage without sustained contact support. To balance retention stability, clean release quality, and residual dexterity, the final mask is selected using a task-aware allocation agent:

$$m^* = \text{AgentSelect}\left(\{m, P_{\text{ret}}(m), P_{\text{clean}}(m), \eta_B(m)\}_{m\in\mathcal{M}}\right) \tag{6}$$

where $\eta_B(m)$ denotes the residual dexterity made available by mask m.

3.3 Dual Residual Stabilizer

After selecting the finger attribution mask m^*, we convert it to joint-level mask $\bar{m}^*$. These masks define action ownership during execution. Task A can only affect the preserved-finger joints, while Task B controls the wrist and the released-finger joints. At the beginning of each composed rollout, after Task A has succeeded, we store the Task-A hand reference q^{ref}. This reference provides the nominal preserved-finger configuration. To compensate for disturbances caused by downstream motion and interaction, we learn a bounded Task-A residual:

$$\Delta q_t^A = \beta_A \odot \tanh\left(\pi_{\text{res}}^A(o_t, m^*)\right), \tag{7}$$

where π_{res}^A is the Task-A residual policy and $\beta_A \in \mathbb{R}_+^{d_q}$ is a per-joint residual bound. The Task-A preservation command is

$$q_t^{A,\text{pres}} = q^{\text{ref}} + \bar{m}^* \odot \Delta q_t^A. \tag{8}$$

We write the corresponding full action vector as

$$a_t^{A,\text{pres}} = \left(\mathbf{0}_6, q_t^{A,\text{pres}}\right). \tag{9}$$

The zero wrist entries are ignored by the final action mask because wrist control is assigned to Task B. For Task-B execution, the frozen downstream policy produces a nominal action $a_t^B = \pi_B(o_t)$.

We then learn a bounded residual correction in the Task-B-owned action subspace:

$$\Delta a_t^B = M_B^* \odot \left[\beta_B \odot \tanh\left(\pi_{\text{res}}^B(o_t, a_t^B, m^*)\right)\right], \tag{10}$$

where π_{res}^B is the Task-B residual policy and $\beta_B \in \mathbb{R}_+^{d_a}$ is a per-action-dimension residual bound. The Task-B execution command is

$$a_t^{B,\text{exec}} = a_t^B + \Delta a_t^B. \tag{11}$$

The final composed action is assembled by the fixed ownership masks:

$$a_t^{\text{comp}} = M_A^* \odot a_t^{A,\text{pres}} + M_B^* \odot a_t^{B,\text{exec}}. \tag{12}$$

Both residual modules are initialized near zero. Therefore, before residual training, the composed policy executes the stored Task-A reference on preserved fingers and the frozen Task-B policy on the wrist and released fingers. During training, m^*, π_A, and π_B remain fixed, and only π_{res}^A and π_{res}^B are optimized.

4 Experiments

We evaluate on 16 composite dexterous manipulation tasks. We first describe the experimental setup in Sec. 4.1 and then compare with the representative policy-composition method in Sec. 4.2, demonstrating that finger attribution and dual-residual stabilizer consistently reuse the base policy and improve composite success. Sec. 4.3 presents a series of diagnostic and ablation studies, including base-policy preservation analysis, failure-mode analysis, and component ablations, to better understand the role of each module in successful policy composition.

4.1 Experimental Setup

Simulation Environments. All simulation experiments are conducted in Isaac Lab [37] using a Shadow Hand [38]. We instantiate Task A with four object-retention skills: GraspBall, PourMug, PickCan, and PickStick. We instantiate Task B with four interaction skills: OpenDoor, PushButton,

Table 1: **Main comparison on 16 composite tasks.** Composite success rate (%), reported as mean $\pm$ standard deviation over eight seeds. "Decomp." denotes Decomposed Action Space, "Res." denotes Residual Learning, and "Ours-ZS" denotes the zero-shot DexCompose variant without the Task-B Residual.

		Composite Success Rate (%)				
First Task	Second Task	Frozen	Decomp.	Res.	Ours-ZS	Ours
GraspBall	OpenDoor	$0.00_{\pm 0.00}$	$38.94_{\pm 2.43}$	$66.35_{\pm 2.58}$	$73.56_{\pm 6.94}$	$\mathbf{82.69}_{\pm 8.33}$
	PushButton	$7.69_{\pm 1.92}$	$42.31_{\pm 2.16}$	$74.04_{\pm 2.11}$	$76.92_{\pm 4.69}$	$\mathbf{85.58}_{\pm 7.50}$
	OpenMicrowave	$0.00_{\pm 0.00}$	$34.62_{\pm 2.57}$	$63.46_{\pm 2.73}$	$64.66_{\pm 7.70}$	$\mathbf{73.08}_{\pm 9.25}$
	TurnOnSwitch	$0.00_{\pm 0.00}$	$31.73_{\pm 2.68}$	$55.29_{\pm 3.08}$	$63.94_{\pm 8.33}$	$\mathbf{71.88}_{\pm 8.93}$
PourMug	OpenDoor	$0.00_{\pm 0.00}$	$36.54_{\pm 2.47}$	$58.65_{\pm 2.81}$	$66.59_{\pm 8.78}$	$\mathbf{75.72}_{\pm 8.08}$
	PushButton	$9.13_{\pm 2.11}$	$40.87_{\pm 2.24}$	$71.63_{\pm 2.17}$	$78.61_{\pm 6.61}$	$\mathbf{85.58}_{\pm 7.28}$
	OpenMicrowave	$0.00_{\pm 0.00}$	$32.21_{\pm 2.61}$	$54.81_{\pm 2.94}$	$52.40_{\pm 7.67}$	$\mathbf{62.74}_{\pm 8.21}$
	TurnOnSwitch	$0.00_{\pm 0.00}$	$30.29_{\pm 2.72}$	$50.96_{\pm 3.16}$	$55.29_{\pm 7.48}$	$\mathbf{64.90}_{\pm 9.89}$
PickCan	OpenDoor	$0.00_{\pm 0.00}$	$35.58_{\pm 2.51}$	$56.73_{\pm 2.84}$	$72.84_{\pm 6.68}$	$\mathbf{79.09}_{\pm 6.04}$
	PushButton	$8.65_{\pm 2.08}$	$39.90_{\pm 2.27}$	$66.83_{\pm 2.46}$	$77.64_{\pm 5.08}$	$\mathbf{85.34}_{\pm 6.92}$
	OpenMicrowave	$0.00_{\pm 0.00}$	$31.25_{\pm 2.64}$	$53.85_{\pm 2.88}$	$64.42_{\pm 11.09}$	$\mathbf{72.36}_{\pm 11.82}$
	TurnOnSwitch	$0.00_{\pm 0.00}$	$29.81_{\pm 2.74}$	$55.29_{\pm 2.67}$	$58.41_{\pm 6.55}$	$\mathbf{65.38}_{\pm 6.81}$
PickStick	OpenDoor	$0.00_{\pm 0.00}$	$83.17_{\pm 1.44}$	$71.15_{\pm 2.17}$	$78.37_{\pm 5.63}$	$\mathbf{86.30}_{\pm 5.78}$
	PushButton	$21.15_{\pm 3.27}$	$84.13_{\pm 1.31}$	$76.44_{\pm 1.94}$	$79.57_{\pm 5.79}$	$\mathbf{86.54}_{\pm 4.84}$
	OpenMicrowave	$0.00_{\pm 0.00}$	$76.92_{\pm 1.86}$	$61.54_{\pm 2.84}$	$77.88_{\pm 6.69}$	$\mathbf{87.26}_{\pm 7.81}$
	TurnOnSwitch	$2.88_{\pm 1.44}$	$61.54_{\pm 2.39}$	$48.56_{\pm 3.22}$	$66.59_{\pm 6.17}$	$\mathbf{74.52}_{\pm 6.10}$
Mean		$3.09_{\pm 0.72}$	$45.61_{\pm 1.94}$	$61.60_{\pm 2.31}$	$69.23_{\pm 1.60}$	$\mathbf{77.43}_{\pm 2.64}$

OpenMicrowave, and TurnOnSwitch. This produces $4 \times 4 = 16$ composite task combinations. All task combinations follow the same evaluation protocol.

Datasets. For each base task, we collect 50 human demonstrations to train the corresponding base policy. During Dual Stabilizer training, we additionally collect 4096 held states for each Task A skill to support stabilization and cross-task composition.

Evaluation Metrics. We primarily evaluate performance using the *Composition Success Rate*, which measures whether both tasks in a composed behavior are successfully completed within a rollout. A rollout is considered successful only if the Task A object remains retained throughout execution and the Task B interaction task succeeds, i.e., $S_{\text{comp}} = S_A \wedge S_B$, where S_A denotes Task A preservation success and S_B denotes Task B success. For each task composition, we evaluate 32×8 rollouts, corresponding to 32 rollouts over 8 random seeds, and report the average success rate across seeds.

4.2 Comparison with Baselines

We compare DexCompose against four baselines that represent different strategies for policy composition. **Frozen grasp** sequentially executes Task A and Task B, freezing the fingers during Task B to preserve the grasp established in Task A. **Decomposed Action Space** composes the two policies by decomposing the overall action space into predefined subspaces. **Residual Learning** augments the combined policy with a learned residual correction. **Ours-ZS** introduces finger allocation together with the Task A stabilizer while directly applying the frozen Task B policy. **Ours** combines finger attribution with dual residual stabilization by learning a Task B residual under the finger assignment. Detailed baseline implementations are provided in the Appendix B.5.

Table 3 reports composite success for all methods. Direct policy chaining fails almost completely, indicating that directly linking two independently trained policies introduces severe interference between their policy distributions during execution. Both the decomposed baseline and the residual learning method substantially improve over direct chaining. However, their performance remains unstable across different task combinations and still perform poorly on interaction-heavy tasks. Our full method achieves the best performance across all task combinations, reaching 77.4% mean composite success and outperforming the strongest baseline by 15.8 percentage points, with the largest gains observed on challenging tasks such as OpenMicrowave and TurnOnSwitch. These consistent

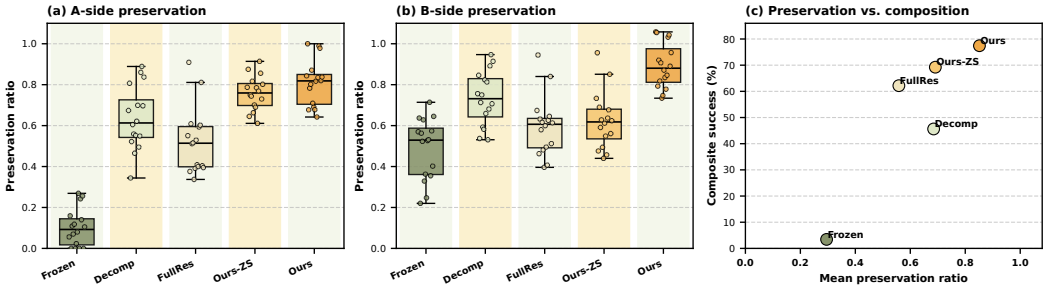

Figure 3: Base-policy preservation analysis. We measure A-side and B-side preservation ratios to evaluate whether composite policies retain the capabilities of the original base policies. Frozen-Grasp preserves part of the Task-B behavior but fails to maintain Task A, while Ours achieves high preservation on both sides.

improvements across all task families demonstrate the effectiveness of our DexCompose framework in maintaining stable multi-task manipulation during sequential policy execution.

4.3 Ablation and Diagnostic Analysis

We conduct ablation and diagnostic studies to isolate the contribution of each component and understand failure modes.

Base-policy preservation. A key point in policy composition is that combining multiple tasks should not significantly degrade the performance of the original base policies. Figure 3 evaluates this property by measuring how well each method preserves the capabilities of the original base policies during composite execution. Frozen-Grasp obtains very low A-side preservation at 0.102, indicating that the first skill is frequently lost either before or during execution of the second task. However, its B-side preservation remains around 0.489, suggesting that the second-skill policy itself is not destroyed. Decomposed method shows substantial variation across compositions, performing well on easier combinations but remaining brittle when tasks compete for hand resources. In contrast, Ours achieves A-side and B-side preservation ratios of 0.811 and 0.893, respectively, and occupies the upper-right region of the preservation-versus-composition plot, combining high mean preservation with the highest composite success. These results demonstrate that our method not only composes policies effectively, but also preserves the original task capabilities as much as possible.

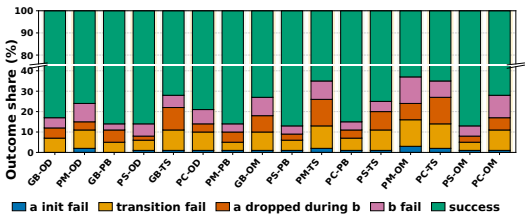

Figure 4: Failure-mode breakdown across task combinations.

Failure-mode analysis. Across all task combinations, the distribution of failures varies significantly depending on the second task. Transition failures are the dominant failure source, indicating that the transition stage is the most challenging and that the stabilizer is critical for maintaining grasp stability. TurnOnSwitch causes more preservation failures, including object drops and *B-fail* outcomes, due to grasp perturbations during switch contact. In contrast, OpenMicrowave exhibits more *B-fail* cases without large increases in drops, suggesting that actuating the microwave handle under constrained grasp configurations is itself difficult. Pick-and-place style second tasks are comparatively stable, producing fewer preservation failures and higher overall success rates. Overall, the results show that task-specific interaction dynamics strongly influence dual-task execution robustness.

Component ablations. Table 2 isolates the contribution of each module. Removing the Task-A residual causes the most severe degradation, confirming it as the essential mechanism for preserving the first-task outcome. Removing Finger Allocation and Action Masking each reduce success substantially, showing that explicit ownership assignment is critical when the two tasks compete for hand resources. The Action Masking result is particularly informative: training a residual that writes to all action dimensions, as in conventional residual learning [30, 27], performs much worse than our masked variant, confirming that structural ownership enforcement is strongly beneficial for two-policy composition. The Task-B residual and transition stage have smaller but meaningful

Table 2: **Component ablation on 16 composite tasks.** Composite success rate (%), reported as mean ± standard deviation over eight seeds. Ours-ZS denotes the zero-shot DexCompose variant without the Task-B Residual. The remaining columns remove one component from the full DexCompose method: −Finger removes Finger Allocation, −Task-A removes the Task-A Residual, −Mask removes Action Masking, and −Trans. removes the Transition Stage.

		Composite Success Rate (%)					
First Task	Second Task	Ours	Ours-ZS	−Finger	−Task-A	−Mask	−Trans.
GraspBall	OpenDoor	**82.69**$_{\pm 8.33}$	73.56$_{\pm 6.94}$	73.80$_{\pm 7.16}$	3.85$_{\pm 3.86}$	67.07$_{\pm 8.23}$	75.00$_{\pm 7.97}$
	PushButton	**85.58**$_{\pm 7.50}$	76.92$_{\pm 4.69}$	70.43$_{\pm 9.43}$	3.37$_{\pm 3.71}$	68.75$_{\pm 8.37}$	79.33$_{\pm 7.61}$
	OpenMicrowave	**73.08**$_{\pm 9.25}$	64.66$_{\pm 7.70}$	43.51$_{\pm 13.29}$	0.96$_{\pm 1.50}$	56.73$_{\pm 10.66}$	64.90$_{\pm 9.73}$
	TurnOnSwitch	**71.88**$_{\pm 8.93}$	63.94$_{\pm 8.33}$	54.33$_{\pm 8.80}$	0.96$_{\pm 1.97}$	53.37$_{\pm 7.49}$	63.22$_{\pm 7.56}$
PourMug	OpenDoor	**75.72**$_{\pm 8.08}$	66.59$_{\pm 8.78}$	62.02$_{\pm 8.64}$	15.38$_{\pm 5.48}$	56.97$_{\pm 6.65}$	64.90$_{\pm 7.88}$
	PushButton	**85.58**$_{\pm 7.28}$	78.61$_{\pm 6.61}$	62.74$_{\pm 8.70}$	17.79$_{\pm 6.55}$	67.55$_{\pm 6.82}$	76.68$_{\pm 6.07}$
	OpenMicrowave	**62.74**$_{\pm 8.21}$	52.40$_{\pm 7.67}$	37.02$_{\pm 6.85}$	13.46$_{\pm 5.62}$	45.43$_{\pm 6.58}$	55.05$_{\pm 7.71}$
	TurnOnSwitch	**64.90**$_{\pm 9.89}$	55.29$_{\pm 7.48}$	40.62$_{\pm 9.29}$	12.02$_{\pm 5.54}$	44.23$_{\pm 7.09}$	55.77$_{\pm 8.54}$
PickCan	OpenDoor	**79.09**$_{\pm 6.04}$	72.84$_{\pm 6.68}$	57.21$_{\pm 11.30}$	14.66$_{\pm 5.32}$	60.82$_{\pm 8.43}$	70.43$_{\pm 6.82}$
	PushButton	**85.34**$_{\pm 6.92}$	77.64$_{\pm 5.08}$	56.49$_{\pm 10.56}$	16.59$_{\pm 6.55}$	62.26$_{\pm 10.48}$	77.64$_{\pm 7.20}$
	OpenMicrowave	**72.36**$_{\pm 11.82}$	64.42$_{\pm 11.09}$	30.29$_{\pm 10.16}$	11.78$_{\pm 5.87}$	51.68$_{\pm 10.49}$	62.02$_{\pm 9.95}$
	TurnOnSwitch	**65.38**$_{\pm 6.81}$	58.41$_{\pm 6.55}$	32.21$_{\pm 6.55}$	11.54$_{\pm 4.84}$	45.43$_{\pm 7.72}$	57.93$_{\pm 6.20}$
PickStick	OpenDoor	**86.30**$_{\pm 5.78}$	78.37$_{\pm 5.63}$	73.32$_{\pm 8.52}$	5.53$_{\pm 5.13}$	71.88$_{\pm 6.38}$	77.40$_{\pm 4.45}$
	PushButton	**86.54**$_{\pm 4.84}$	79.57$_{\pm 5.79}$	70.91$_{\pm 9.23}$	6.25$_{\pm 4.23}$	68.27$_{\pm 7.95}$	78.12$_{\pm 4.94}$
	OpenMicrowave	**87.26**$_{\pm 7.81}$	77.88$_{\pm 6.69}$	54.33$_{\pm 8.61}$	3.37$_{\pm 2.98}$	68.27$_{\pm 6.73}$	77.64$_{\pm 8.45}$
	TurnOnSwitch	**74.52**$_{\pm 6.10}$	66.59$_{\pm 6.17}$	53.12$_{\pm 10.36}$	2.88$_{\pm 4.13}$	57.45$_{\pm 7.71}$	68.27$_{\pm 7.09}$
Average	–	**77.43**$_{\pm 2.64}$	69.23$_{\pm 1.60}$	54.52$_{\pm 2.09}$	9.13$_{\pm 1.29}$	59.13$_{\pm 1.75}$	69.02$_{\pm 2.07}$

effects, primarily improving execution refinement and stage-switching stability. Overall, the ablation reveals a clear hierarchy: Task-A preservation is essential, finger allocation and action masking reduce interference, and the Task-B residual with the transition stage further improves robustness.

Agent-based mask selection. We compare the agent-based mask selector with a heuristic baseline that selects the mask with the highest clean-release rate under a minimum finger-release constraint. In contrast, the LLM additionally reasons about grasp stability and downstream finger functionality. As shown in Table 3, the LLM consistently achieves higher success rates on task pairs where the two methods choose different masks, with especially large gains on PickStick tasks. These results suggest that language-based reasoning provides a more reliable balance between object retention and downstream dexterity than metric-only selection.

Table 3: Heuristic vs. LLM mask selection.

Task Pair	Heur.	LLM
PickStick+Door	62.5	**86.3**
PickStick+Button	65.4	**86.5**
PickCan+Micro.	68.3	**72.4**
PourMug+Switch	62.0	**64.9**
Mean	64.5	**77.5**

5 Conclusion

We presented DexCompose, a framework for composing pretrained dexterous manipulation policies by treating embodiment redundancy as a reusable resource. DexCompose identifies redundant action dimensions, assigns structured action ownership through finger-aware masking, and applies dual residual stabilizers within the allocated subspaces. Our results show that structured action ownership reduces cross-task interference and is critical for successful composite manipulation. More broadly, DexCompose points toward a scalable framework for reusable dexterous manipulation, enabling learned skills to be flexibly composed without requiring task-specific retraining of the base policies..

Limitations and Future Works. Our current framework focuses on composing two sequential skills at a time. Extending the framework to longer-horizon compositions involving multiple interacting skills remains an important direction for future work.

References

[1] Y. Li, B. Liu, Y. Geng, P. Li, Y. Yang, Y. Zhu, T. Liu, and S. Huang. Grasp multiple objects with one hand. *IEEE Robotics and Automation Letters*, 9(5):4027–4034, 2024. doi:10.1109/LRA.2024.3374190.

[2] H. Jiang, Y. Wu, Y. Wang, G. S. Sukhatme, and D. Seita. Concurrent prehensile and nonprehensile manipulation: A practical approach to multi-stage dexterous tasks, 2026.

[3] E. Foong, Y. Li, H. Jiang, G. S. Sukhatme, and D. Seita. HANDFUL: Sequential grasp-conditioned dexterous manipulation with resource awareness, 2026.

[4] R. Wang, J. Zhang, J. Chen, Y. Xu, P. Li, T. Liu, and H. Wang. DexGraspNet: A large-scale robotic dexterous grasp dataset for general objects based on simulation. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pages 11359–11366, 2023. doi:10.1109/ICRA48891.2023.10160982.

[5] Y. Xu, W. Wan, J. Zhang, H. Liu, Z. Shan, H. Shen, R. Wang, H. Geng, Y. Weng, J. Chen, T. Liu, L. Yi, and H. Wang. UniDexGrasp: Universal robotic dexterous grasping via learning diverse proposal generation and goal-conditioned policy. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4737–4746, 2023.

[6] W. Wan, H. Geng, Y. Liu, Z. Shan, Y. Yang, L. Yi, and H. Wang. UniDexGrasp++: Improving dexterous grasping policy learning via geometry-aware curriculum and iterative generalist-specialist learning. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 3891–3902, 2023.

[7] I. Popov, N. Heess, T. Lillicrap, R. Hafner, G. Barth-Maron, M. Vecerik, T. Lampe, Y. Tassa, T. Erez, and M. Riedmiller. Data-efficient deep reinforcement learning for dexterous manipulation. In *International Conference on Learning Representations*, 2018.

[8] A. Rajeswaran, V. Kumar, A. Gupta, G. Vezzani, J. Schulman, E. Todorov, and S. Levine. Learning complex dexterous manipulation with deep reinforcement learning and demonstrations. In *Proceedings of Robotics: Science and Systems (RSS)*, 2018.

[9] OpenAI, M. Andrychowicz, B. Baker, M. Chociej, R. Józefowicz, B. McGrew, J. Pachocki, A. Petron, M. Plappert, G. Powell, A. Ray, J. Schneider, S. Sidor, J. Tobin, P. Welinder, L. Weng, and W. Zaremba. Learning dexterous in-hand manipulation. *The International Journal of Robotics Research*, 39(1):3–20, 2020. doi:10.1177/0278364919887447.

[10] I. Akkaya, M. Andrychowicz, M. Chociej, M. Litwin, B. McGrew, A. Petron, A. Paino, M. Plappert, G. Powell, R. Ribas, J. Schneider, N. Tezak, J. Tworek, P. Welinder, L. Weng, Q. Yuan, W. Zaremba, and L. Zhang. Solving rubik's cube with a robot hand. *arXiv preprint arXiv:1910.07113*, 2019.

[11] Y. Qin, Y.-H. Wu, S. Liu, H. Jiang, R. Yang, Y. Fu, and X. Wang. Dexmv: Imitation learning for dexterous manipulation from human videos. In *Computer Vision – ECCV 2022*, pages 570–587. Springer, 2022. doi:10.1007/978-3-031-19842-7_33.

[12] P. Mandikal and K. Grauman. Dexvip: Learning dexterous grasping with human hand pose priors from video. In *Conference on Robot Learning*, 2021.

[13] S. P. Arunachalam, S. Silwal, B. Evans, and L. Pinto. Dexterous imitation made easy: A learning-based framework for efficient dexterous manipulation. In *2023 IEEE International Conference on Robotics and Automation*, 2023.

[14] Z. Jiang, Y. Xie, K. Lin, Z. Xu, W. Wan, A. Mandlekar, L. Fan, and Y. Zhu. Dexmimicgen: Automated data generation for bimanual dexterous manipulation via imitation learning. In *2025 IEEE International Conference on Robotics and Automation*, 2025.

[15] C. Chi, S. Feng, Y. Du, Z. Xu, E. Cousineau, B. Burchfiel, and S. Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Proceedings of Robotics: Science and Systems (RSS)*, 2023.

[16] Y. Ze, G. Zhang, K. Zhang, C. Hu, M. Wang, and H. Xu. 3d diffusion policy: Generalizable visuomotor policy learning via simple 3d representations. *arXiv preprint arXiv:2403.03954*, 2024.

[17] C. Bao, H. Xu, Y. Qin, and X. Wang. DexArt: Benchmarking generalizable dexterous manipulation with articulated objects. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 21190–21200, 2023.

[18] R. Wang, J. Zhang, J. Chen, Y. Xu, P. Li, T. Liu, and H. Wang. Dexgraspnet: A large-scale robotic dexterous grasp dataset for general objects based on simulation. In *2023 IEEE International Conference on Robotics and Automation*, pages 11359–11366, 2023.

[19] J. Zhang, H. Liu, D. Li, X. Yu, H. Geng, Y. Ding, J. Chen, and H. Wang. Dexgraspnet 2.0: Learning generative dexterous grasping in large-scale synthetic cluttered scenes. In *Proceedings of the 8th Conference on Robot Learning*, volume 270 of *Proceedings of Machine Learning Research*. PMLR, 2025.

[20] Y. Chen, C. Wang, L. Fei-Fei, and K. Liu. Sequential dexterity: Chaining dexterous policies for long-horizon manipulation. In J. Tan, M. Toussaint, and K. Darvish, editors, *Proceedings of The 7th Conference on Robot Learning*, volume 229 of *Proceedings of Machine Learning Research*, pages 3809–3829. PMLR, 2023.

[21] K. Pertsch, Y. Lee, and J. J. Lim. Accelerating reinforcement learning with learned skill priors. In *Proceedings of the 2020 Conference on Robot Learning*, volume 155 of *Proceedings of Machine Learning Research*, pages 188–204. PMLR, 2021.

[22] A. Singh, H. Liu, G. Zhou, A. Yu, N. Rhinehart, and S. Levine. Parrot: Data-driven behavioral priors for reinforcement learning. In *International Conference on Learning Representations*, 2021.

[23] S. Nasiriany, H. Liu, and Y. Zhu. Augmenting reinforcement learning with behavior primitives for diverse manipulation tasks. In *2022 IEEE International Conference on Robotics and Automation*, pages 7477–7484, 2022.

[24] S. Nasiriany, T. Gao, A. Mandlekar, and Y. Zhu. Learning and retrieval from prior data for skill-based imitation learning. In *Proceedings of the 6th Conference on Robot Learning*, volume 205 of *Proceedings of Machine Learning Research*, pages 2181–2204. PMLR, 2023.

[25] S. He, Z. Shangguan, K. Wang, Y. Gu, Y. Fu, Y. Fu, and D. Seita. Sequential multi-object grasping with one dexterous hand. *arXiv preprint arXiv:2503.09078*, 2025.

[26] H. Lu, Y. Dong, Z. Weng, F. T. Pokorny, J. Lundell, and D. Kragic. Grasping a handful: Sequential multi-object dexterous grasp generation. *IEEE Robotics and Automation Letters*, 10(11):11880–11887, 2025. doi:10.1109/LRA.2025.3614051.

[27] T. Silver, K. Allen, J. Tenenbaum, and L. Kaelbling. Residual policy learning. *arXiv preprint arXiv:1812.06298*, 2018.

[28] A. Ranjbar, N. A. Vien, H. Ziesche, J. Boedecker, and G. Neumann. Residual feedback learning for contact-rich manipulation tasks with uncertainty. In *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 2383–2390, 2021.

[29] Y. Shi, Z. Chen, H. Liu, S. Riedel, C. Gao, Q. Feng, J. Deng, and J. Zhang. Proactive action visual residual reinforcement learning for contact-rich tasks using a torque-controlled robot. In *2021 IEEE International Conference on Robotics and Automation*, pages 765–771, 2021.

[30] T. Johannink, S. Bahl, A. Nair, J. Luo, A. Kumar, M. Loskyll, J. A. Ojea, E. Solowjow, and S. Levine. Residual reinforcement learning for robot control. In *2019 International Conference on Robotics and Automation (ICRA)*, pages 6023–6029. IEEE, 2019. doi:10.1109/ICRA.2019.8794127.

[31] C. Schaff and M. R. Walter. Residual policy learning for shared autonomy. In *Proceedings of Robotics: Science and Systems (RSS)*, 2020.

[32] M. Alakuijala, G. Dulac-Arnold, J. Mairal, J. Ponce, and C. Schmid. Residual reinforcement learning from demonstrations. *arXiv preprint arXiv:2106.08050*, 2021.

[33] C. Chi, B. Burchfiel, E. Cousineau, S. Feng, and S. Song. Iterative residual policy for goal-conditioned dynamic manipulation of deformable objects. In *Proceedings of Robotics: Science and Systems (RSS)*, 2022.

[34] K. Rana, M. Xu, B. Tidd, M. Milford, and N. Sünderhauf. Residual skill policies: Learning an adaptable skill-based action space for reinforcement learning for robotics. In *Proceedings of the 6th Conference on Robot Learning*, volume 205 of *Proceedings of Machine Learning Research*, pages 2095–2104. PMLR, 2023.

[35] L. L. Ankile, A. Simeonov, I. Shenfeld, M. Torne, and P. Agrawal. From imitation to refinement: Residual rl for precise assembly. In *2025 IEEE International Conference on Robotics and Automation*, 2025.

[36] K. Li, P. Li, T. Liu, Y. Li, and S. Huang. Maniptrans: Efficient dexterous bimanual manipulation transfer via residual learning. *arXiv preprint arXiv:2503.21860*, 2025.

[37] NVIDIA. Isaac lab, 2024. URL `https://github.com/isaac-sim/IsaacLab`. Robotics reinforcement learning and simulation framework built on NVIDIA Isaac Sim.

[38] Shadow Robot Company. Shadow dexterous hand, 2024. URL `https://www.shadowrobot.com/dexterous-hand-series/`. 24-DoF anthropomorphic robotic hand platform.

[39] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

A Base Policy Training and Evaluation Protocol

We train one diffusion-based policy for each primitive dexterous manipulation task using replayed successful human demonstrations collected in Isaac Lab. The eight base policies correspond to four object-retention skills (GraspBall, PickStick, PickCan, PourMug) and four downstream interaction skills (OpenDoor, PushButton, OpenMicrowave, TurnOnSwitch).

Observation and action spaces. All policies operate on low-dimensional proprioceptive and task-state observations. The observation consists of normalized hand joint positions together with task-specific geometric features such as object-relative poses, articulated-object joint states, and palm-to-object offsets. Depending on the task, the observation dimension ranges from 32 to 37.

The action space is shared across all tasks and consists of 28-dimensional Floating Shadow Hand joint-position commands:

$$a_t = (p_t, r_t, q_t),$$

where $p_t \in \mathbb{R}^3$ denotes wrist translation, $r_t \in \mathbb{R}^3$ denotes wrist rotation, and q_t denotes the 22 finger-joint position targets of the Shadow Hand.

Diffusion policy architecture. All base policies share the same conditional diffusion architecture. Given the latest two observations ($obs\_horizon = 2$), the policy predicts a horizon of 32 future actions ($action\_horizon = 32$).

We use a FiLM-conditioned 1D UNet diffusion backbone. The observation history is flattened and encoded with an MLP:

$$\text{Linear} \rightarrow \text{Mish} \rightarrow \text{Linear},$$

and injected into the diffusion timestep embedding through FiLM-style conditioning. The denoising backbone uses channel widths $(256, 512, 1024)$ with residual temporal blocks and Mish activations.

Diffusion training uses a DDPM objective with 100 diffusion timesteps and the `squaredcos_cap_v2` noise schedule.

Training details. Policies are trained with behavior cloning on replayed successful demonstrations. Replay trajectories are segmented into overlapping observation-action windows of shape:

$$obs \in \mathbb{R}^{2\times d_{obs}}, \qquad action \in \mathbb{R}^{32\times 28}.$$

All policies use AdamW optimization with batch size 256, learning rate 1×10^{-4}, 2000-step linear warmup followed by cosine decay, EMA checkpoint averaging, and gradient clipping with global norm 1.0.

Inference protocol. At evaluation time, the policy samples an action chunk by iterative denoising from Gaussian noise. We execute only the first several actions from the predicted sequence and then replan in a receding-horizon manner until task success or timeout.

Standalone base-policy performance. Table 4 reports the standalone success rate of all pretrained base policies before policy composition.

Task success criteria. For object-retention tasks, success is defined using task-specific geometric conditions. GraspBall succeeds when the object is within a threshold distance of the target pose. PickStick, PickCan, and PourMug additionally require the object to be lifted above a minimum height while satisfying task-specific orientation constraints.

For downstream interaction tasks, OpenDoor and OpenMicrowave succeed when the articulated joint angle exceeds a predefined opening threshold. PushButton succeeds when the button displacement exceeds a target travel distance. TurnOnSwitch succeeds when the switch joint reaches at least 80% of its reachable joint range.

Table 4: Standalone success rate (%) of pretrained base policies. Results are evaluated using the corresponding execute horizon for each task.

Task	Execute Horizon	Success Rate (%)
GraspBall	8	100.0
PickStick	16	100.0
PickCan	16	100.0
PourMug	16	100.0
OpenDoor	24	97.59
OpenMicrowave	24	89.90
PushButton	16	100.0
TurnOnSwitch	30	82.21

For composite-task evaluation, a rollout is considered successful only if the Task-A object remains retained throughout execution and the downstream Task-B interaction succeeds:

$$S_{\text{comp}} = S_A \wedge S_B.$$

All methods and baselines are evaluated under the same success definitions and rollout horizons.

B Learning Procedure and Implementation Details

This appendix provides the complete learning procedure for all trained components of DexCompose, addressing the reproducibility requirements for the allocation mechanism (Section 3.2) and the two residual policies (Section 3.3). All experiments are conducted on a single NVIDIA RTX 4090 GPU.

B.1 PPO Training: Task A Residual Stabilizer

The Task A residual policy π^A_{res} is trained via Proximal Policy Optimization (PPO) [39] to produce bounded joint-space corrections that stabilize the held object against disturbances from base motion and finger release.

Observation space. The observation includes: preserved-finger joint positions and velocities, the previous residual output, object pose and velocity in the palm frame, fingertip-to-object distance errors, binary contact indicators, torque features at the preserved finger joints, the executed base action, and a rollout phase indicator.

Action space. The residual is a Gaussian policy whose mean head is initialized to zero and whose log-standard-deviation is initialized to -1.5. The output is bounded via $\Delta^A_t = b_A \odot \tanh(\pi^A_{\text{res}}(o_t, m^*))$, where b_A is a per-dimension bound vector. Zero-mean initialization ensures that the policy begins at the stored hold reference with no correction.

Reward function. The reward at each timestep penalizes the Euclidean distance between the held object and a target position near the palm center, plus a small penalty on the magnitude of the residual correction to discourage unnecessary adjustments:

$$R^A_t = -\|\mathbf{p}_{\text{obj}} - \mathbf{p}_{\text{palm}}\|_2 - \alpha\|\Delta^A_t\|_2^2, \tag{13}$$

with $\alpha = 0.001$.

Architecture and hyperparameters. The actor and critic share an MLP backbone of $[256, 256, 128]$ units with ELU activations and Prefix Layer Normalization before each hidden layer. Complete hyperparameter settings are provided in Appendix B.7.

Training budget. Training uses 1024 parallel environments with rollout length 24, for 1000 PPO iterations, yielding $24{,}576{,}000$ total environment steps. Wall-clock time is approximately 20 minutes on the RTX 4090.

B.2 PPO Training: Task B Residual

The Task B residual policy π^B_{res} is trained via PPO with the same backbone architecture and hyperparameters as π^A_{res} (see Appendix B.7), with the following differences.

Observation space. In addition to the composite state, which includes Task A held object and preserved finger states, the policy observes the nominal Task B policy output $a^0_{B,t} = \pi_B(s_t)$ and the selected mask m^*. This allows the residual to condition its correction on what the frozen Task B policy would have executed and which action dimensions are available.

Action space. The residual δ^B_t is produced by the same Gaussian actor architecture, bounded, and masked by M_B before execution (Eqs. 17–18). Only action dimensions assigned to Task B are affected.

Reward function. The reward combines Task B progress and Task A preservation:

$$R^B_t = R_{\text{taskB}}(s_t, a_t) - \beta\|\mathbf{p}_{\text{obj}} - \mathbf{p}_{\text{palm}}\|_2 - \alpha\|\delta^B_t\|^2_2, \tag{14}$$

where R_{taskB} is the environment-defined Task B success reward (e.g., door angle for OpenDoor), and the object–palm distance term is identical to that used in Phase 1. We set $\beta = 1.0$ and $\alpha = 0.001$.

Joint Task A fine-tuning. During Task B residual training, the Task A residual π^A_{res} is optionally fine-tuned jointly with a reduced learning rate of 1×10^{-4} while the Task B residual trains at the standard rate (3×10^{-4}). This allows the Task A stabilizer to adapt to the specific disturbance patterns induced by Task B execution without catastrophic forgetting.

Training budget. Training uses 8 parallel environments with rollout length 200, for 1000 PPO iterations, yielding 1,600,000 nominal environment-step slots. Due to per-episode filtering from pre-door termination and early success, the effective number of samples per PPO update varies between approximately 408 and 750. Wall-clock time is approximately 4 hours on the RTX 4090.

B.3 Training Curriculum

The full training curriculum consists of three sequential stages:

1. **Held-state collection (no learning):** For each Task A skill (GraspBall, PourMug, PickCan, PickStick), we collect $N = 4096$ successful held-object states by rolling out the pretrained Task A policy to completion. Each state stores the simulator state, object pose, hand configuration, contact information, and reference hold action. States are sampled uniformly during subsequent release tests.
2. **Phase 1 (Task A residual training):** For each Task A skill, π^A_{res} is trained independently (not yet coupled to a specific Task B) using the reward defined in Appendix B.1. The policy learns to stabilize the held object against randomized base and finger perturbations. After training, π^A_{res} is frozen.
3. **Phase 2 (Finger allocation and Task B residual training):** For each composite task pair:
   (a) The LLM selects a finger mask m^* from the candidate set $\mathcal{M}$ using release-test diagnostics computed from the stored held-state library (Appendix B.8).
   (b) **Release-test evaluation:** For each candidate mask, we run $K = 100$ release-test rollouts. Each rollout restores a held state sampled uniformly from the library and runs the masked replay protocol for T_{test} steps (Appendix B.4).
   (c) With m^* fixed, π^B_{res} is trained with π^A_{res} and π_B frozen (optionally with joint Task A fine-tuning, Appendix B.2).

No joint fine-tuning of the full composite system is performed beyond the optional joint Task A update during Phase 2. The frozen Task B policy π_B is never updated.

B.4 Release-Test Protocol

For each candidate mask $m \in \mathcal{M}$, we perform the following release-test procedure:

1. **Restore held state:** The simulator is reset to a held state s_A^j sampled uniformly from $\mathcal{D}_{\text{hold}}$.
2. **Masked replay:** Fingers in $G_A(m)$ replay the stored Task A hold reference action. Fingers in $G_B(m)$ are driven toward an open or neutral pose following the linear ramp schedule:

$$\lambda_t = \frac{t}{T_{\text{test}}}, \tag{15}$$

   where $T_{\text{test}} = 100$ simulation steps. At $t = 0$, released fingers are at the hold configuration. At $t = T_{\text{test}}$, they reach the fully open pose.
3. **Retention check:** The object is *retained* if it remains within a distance threshold d_{ret} of the palm center and above a drop-height threshold h_{drop} at the end of the test.
4. **Clean-release check:** Released fingers are considered *free* if their contact forces fall below a threshold $F_{\min}$ during the final 20 steps of the test, indicating they no longer provide sustained support.
5. **Aggregation:** We repeat Steps 1–4 for K independent rollouts (default $K = 100$) per candidate mask by resampling held states from $\mathcal{D}_{\text{hold}}$. $P_{\text{ret}}(m)$ and $P_{\text{clean}}(m)$ are computed as the fraction of rollouts passing each check (as defined in the main text).

We use $d_{\text{ret}} = 0.05$ m, $h_{\text{drop}} = 0.03$ m, and $F_{\min} = 0.1$ N.

B.5 Baseline Implementation Details

We provide detailed implementation descriptions for the baselines compared in Section 3.

Frozen grasp ("Frozen" in Table 1). This baseline sequentially executes the frozen Task A policy π_A and frozen Task B policy π_B. After Task A success is detected, we execute π_B while *freezing the grasp-maintaining finger joints* at the final Task A configuration (i.e., those finger action dimensions are held constant during Task B). All non-finger action dimensions (e.g., base and arm) are controlled by π_B. This baseline therefore preserves the grasp by preventing Task B from writing to the frozen finger dimensions, but it does not perform finger allocation, action masking for released fingers, release-test validation, or residual correction.

Direct Combination (naive chain; not shown in Table 1). The frozen Task A policy π_A and frozen Task B policy π_B are executed sequentially without any coordination mechanism. The terminal state of π_A, once Task A success is detected, directly initializes π_B, with no finger allocation, action masking, freezing, or residual correction. Both policies output to the full action space, so π_B overwrites all action dimensions including those that would otherwise maintain the grasp.

Decomposed Action Space. We retrain both base policies with an auxiliary loss that encourages each policy to minimize action magnitudes on dimensions assigned to the other task. Specifically, for a fixed finger allocation mask m, the Task A policy is trained with an additional penalty $\lambda \| M_B \odot a_t^A \|_2^2$ on actions in the Task B subspace, and symmetrically for the Task B policy. The allocation mask for this baseline is fixed per task family. For GraspBall and PourMug, thumb, index, and middle are preserved. For PickCan, index, middle, and ring are preserved. For PickStick, thumb and index are preserved. These assignments are chosen based on the dominant contact pattern observed in single-task rollouts and are not optimized per task pair. The penalty coefficient $\lambda = 0.1$ is tuned on a held-out validation pair. Both policies are retrained for the same number of environment steps as the original single-task policies. During composite execution, actions from both policies are combined via the same hard-mask overwrite as our method (Eq. 13), but neither policy receives residual corrections or release-test validation.

Residual Learning. We adapt the residual RL formulation of Johannink et al. [30] to the two-stage composition setting. A single Q-function and policy are trained to output corrections to the combined nominal action:

$$a_t = \underbrace{M_A \odot a_A^{\text{ref}} + M_B \odot \pi_B(s_t)}_{\text{nominal}} + \delta_t, \tag{16}$$

where $\delta_t \in \mathbb{R}^d$ is an unconstrained residual (no action masking applied to the correction), a_A^{ref} is the stored Task A hold action, and M_A, M_B use the same fixed allocation masks as the Decomposed baseline. The residual policy observes the full composite state and is trained via soft actor-critic with the same composite reward as our Task B residual (Appendix B.2). The critical difference from our method is that δ_t is not masked before execution, so the residual can write to any action dimension, including those assigned to Task A preservation. This baseline therefore isolates the effect of structural action ownership enforcement: it uses residuals but lacks the $M_A \odot M_B = 0$ hard constraint.

Ours (w/o Task-B residual). This ablation uses our full finger allocation pipeline with LLM-based mask selection and release-test validation, and the trained Task A residual stabilizer, but executes the frozen Task B policy directly without residual correction. The Task B nominal action is masked as $M_B \odot \pi_B(s_t)$ and applied to the Task B-owned dimensions. This isolates the contribution of the Task B residual in adapting the frozen policy to the constrained hand state.

B.6 Diagnostic Figures

B.7 Hyperparameters and Compute Resources

We provide the complete hyperparameter settings for reproducibility. All experiments are conducted on a single NVIDIA RTX 4090 with 24 GB of memory.

PPO hyperparameters. Both Task A and Task B residual policies are trained with Proximal Policy Optimization using the same core settings. The learning rate is 3×10^{-4} for standard training and reduced to 1×10^{-4} for joint Task A fine-tuning during Phase 2. The PPO clipping parameter ϵ is set to 0.2, and we use GAE with $\lambda = 0.95$ and a discount factor $\gamma = 0.99$. Each PPO iteration performs 5 update epochs over 4 minibatches. The entropy coefficient is 0.001, the value loss coefficient is 1.0, and gradients are clipped to a maximum norm of 1.0.

Architecture. The actor and critic share an MLP backbone with hidden layers of sizes $[256, 256, 128]$, using ELU activations and Prefix Layer Normalization before each hidden layer. The actor outputs a Gaussian distribution over residual corrections, with the mean head initialized to zero and the log-standard-deviation initialized to -1.5. Both policies use this same architecture.

Task A training budget. The Task A residual stabilizer is trained with 1024 parallel environments, a rollout length of 24 steps per environment, yielding a nominal batch size of $24{,}576$ per PPO iteration and a minibatch size of 6144. Training runs for 1000 PPO iterations, totaling $24{,}576{,}000$ environment steps. Wall-clock time is approximately 20 minutes.

Task B training budget. The Task B residual is trained with 8 parallel environments and a rollout length of 200 steps, giving a nominal batch size of 1600 and a minibatch size of 400. Due to per-episode filtering from pre-door termination and early success, the effective number of samples per update varies between approximately 408 and 750. Training runs for 1000 PPO iterations, yielding $1{,}600{,}000$ nominal environment steps. Wall-clock time is approximately 4 hours.

Reward parameters. The object-to-palm distance is weighted by $\beta = 1.0$ in the Task B composite reward. The residual magnitude penalty uses $\alpha = 0.001$ for both Task A and Task B policies.

Release-test parameters. We collect a library of $N = 4096$ held states per Task A skill. For each candidate finger mask, we run $K = 100$ release-test rollouts by sampling held states uniformly

from this library (with replacement). Each rollout lasts $T_{\text{test}} = 100$ simulation steps. The retention distance threshold is $d_{\text{ret}} = 0.05$ m, the drop-height threshold is $h_{\text{drop}} = 0.03$ m, and the clean-release force threshold is $F_{\min} = 0.1$ N.

Mask allocation. The final mask m^* is selected via a task-aware allocation procedure that uses GPT-5.4 (snapshot `gpt-5.4-2026-03-05`, accessed May 18, 2026) with temperature $T = 0$ for deterministic selection. Full details and a comparison with a heuristic baseline are provided in Appendix B.8.

B.8 LLM-Based Finger Mask Selection

As described in Section 3.2, finger mask selection is performed by a large language model rather than a learned allocation network. We use OpenAI GPT-5.4 (snapshot `gpt-5.4-2026-03-05`, accessed May 18, 2026) with temperature $T = 0$ to ensure deterministic mask selection.

For each task pair, the LLM receives a structured prompt consisting of three components: (1) a natural-language description of the object manipulation in Task A and the downstream interaction required in Task B; (2) a list of candidate masks $\mathcal{M}$ together with their release-test diagnostics, including retention rate $P_{\text{ret}}(m)$, clean-release rate $P_{\text{clean}}(m)$, released-finger count, contact-force statistics, and downstream availability score; and (3) an instruction specifying the desired trade-off between grasp stability and downstream dexterity.

The instruction template provided to the LLM is:

"You are selecting a finger-retention mask for a dual-task robotic manipulation problem. The robot hand has five fingers and must continue stabilizing the object from Task A while freeing sufficient fingers to execute Task B. Each candidate mask specifies which fingers remain in contact with the object and which fingers are released for downstream interaction.
A good mask should satisfy three objectives simultaneously: (1) maintain stable object retention, (2) allow clean finger release without disturbing the grasp, and (3) preserve dexterous and anatomically appropriate fingers for Task B.
When evaluating candidates, consider not only the numerical metrics but also the semantic suitability of the finger assignment. For example, thumb-index opposition is generally preferable for stabilizing slender cylindrical objects, while releasing the thumb may significantly reduce grasp stability even if the clean-release score is high. Avoid overly conservative masks that release too few fingers and overly aggressive masks that sacrifice object stability.
Select the single mask that provides the best overall trade-off between grasp stability and downstream manipulation capability."

The LLM outputs the selected mask index, which is directly parsed and used as the execution mask m^* during masked policy rollout. No reinforcement learning or fine-tuning is used for this allocation stage. Since all candidate masks and diagnostics are precomputed, the selection process is fully reproducible under deterministic decoding.

Heuristic baseline. To evaluate whether language-based reasoning is necessary, we additionally compare against a heuristic baseline that selects the mask with the highest clean-release rate subject to a minimum-release constraint:

$$m_{\text{heur}} = \arg\max_{m \in \mathcal{M}} P_{\text{clean}}(m) \quad \text{s.t.} \quad 5 - \|m\|_0 \geq 2. \tag{17}$$

The constraint ensures that at least two fingers are released for Task B execution. Unlike the LLM selector, this heuristic relies purely on release-test statistics and does not incorporate reasoning about grasp anatomy, object geometry, or downstream interaction requirements.